

ISYS1055/1057/3412 (Practical) Database Concepts

Assessment 2: SQL Programming and Normalization

Student Name: Syeda Shabnam Khan

Student ID: 4020189

Program: Master of Data Science

Part A

Problem 1: Functional Dependencies

Customer (CustNo, CustName, Phone, Address, DateOfBirth, EmailAddress)

- CustNo -> CustName
- CustNo -> Phone
- CustNo -> Address
- CustNo -> DateOfBirth
- CustNo -> EmailAddress

Account(AccNo, CustNo*, CustName*, EmpNo*)

- AccNo -> CustNo
- AccNo -> CustName
- AccNo -> EmpNo
- CustNo -> CustName
- CustNo -> EmpNo

Employee (EmpNo, RegisterBranchBSB, BranchAddress)

- EmpNo -> RegisterBranchBSB
- EmpNo -> BranchAddress
- RegisterBranchBSB -> BranchAddress

AccType (TypeID, TypeName)

- TypeID -> TypeName

CustomerACCType (CustNo*, TypeID*, CustName*, TypeName*)

- CustNo -> CustName

- TypeID -> TypeName
- No functional dependency between CustNo and TypeID

Problem 2: Identifying keys and Normal forms

Customer (CustNo, CustName, Phone, Address, DateOfBirth, EmailAddress)

- Primary Key is CustNo
- No foreign key
- No other candidate key.
- This relation is in 3NF.
- It is in 3NF.
- All attributed depends on the key.
- There are no composite keys, so no partial dependencies.
- No non-key attribute depend on any other non-key attribute.

Account(AccNo, CustNo*, CustName*, EmpNo*)

- Primary key is AccNo
- AccNo and CustNo together is a candidate key but it is not minimal.
- CustNo, CustName and EmpNo are foreign keys.
- This relation is in 2NF.
- Everything depends on the key.
- There are no composite keys.
- It has transitive dependency. Non key attributes CustName and EmpNo depend on non-key attribute CustNo.

Employee (EmpNo, RegisterBranchBSB, BranchAddress)

- EmpNo is the primary key.
- No foreign key.
- No other candidate key.
- It is in 2NF.
- Everything depends on the key.
- There are no composite keys.
- It has transitive dependency. Non-key BranchAddress depends on non-key RegisterBranchBSB

AccType (TypeID, TypeName)

- Primary key is TypeID.
- No foreign key.
- TypeName is a candidate key.
- It is in 3NF
- All attributed depends on the key.
- There are no composite keys, so no partial dependencies.
- No non-key attribute depends on any other non-key attribute.

CustomerACCType (CustNo*, TypeID*, CustName*, TypeName*)

- Primary key is CustNo and TypeID together. It is a composite key.
- CustNo, CustName, TypeID and TypeName are all foreign keys.
- CustNo and TypeName together is a candidate key.
- It is in 1NF.
- It has partial dependency. CustName and TypeName only depend on part of the key.
- As it is not in 2NF, it cannot be in 3NF.

Problem 3: Decomposition

Account(AccNo, CustNo*, CustName*, EmpNo*)

- AccNo -> CustNo, CustName, EmpNo
- CustNo -> CustName, EmpNo

~~Account1(AccNo, CustNo*)~~

Account(AccNo, CustNo*)

~~Account2(CustNo, CustName, EmpNo)~~

ServicedBy(CustNo, CustName, EmpNo)

Employee (EmpNo, RegisterBranchBSB, BranchAddress)

- EmpNo -> RegisterBranchBSB, BranchAddress
- RegisterBranchBSB -> BranchAddress

~~Employee1(EmpNo, RegisterBranchBSB*)~~

Employee(EmpNo, RegisterBranchBSB*)

~~Employee2(RegisterBranchBSB, BranchAddress)~~

Branch(RegisterBranchBSB, BranchAddress)

CustomerACCType (CustNo*, TypeID*, CustName*, TypeName*)

- CustNo -> CustName
- TypeID -> TypeName

~~CustomerAccType1(CustNo*, TypeID*)~~

CustomerAccType(CustNo*, TypeID*)

~~CustomerAccType2(CustNo, CustName)~~

Customer(CustNo, CustName)

~~CustomerAccType3(TypeID, TypeName)~~

AccType(TypeID, TypeName)

Problem 4: Final Schema

Customer (CustNo, CustName, Phone, Address, DateOfBirth, EmailAddress)

~~AccType (TypeID, TypeName)~~

Account(AccNo, CustNo*)

~~ServicedBy(CustNo, CustName, EmpNo)~~

Employee(EmpNo, RegisterBranchBSB*)

Branch(RegisterBranchBSB, BranchAddress)

CustomerAccType(CustNo*, TypeID*)

~~Customer(CustNo, CustName)~~

~~AccType(TypeID, TypeName)~~

After merging schema with same primary key:

Customer (CustNo, CustName, Phone, Address, DateOfBirth, EmailAddress, EmpNo*)

AccType(TypeID, TypeName)

Account(AccNo, CustNo*)

Employee(EmpNo, RegisterBranchBSB*)

Branch(RegisterBranchBSB, BranchAddress)

CustomerAccType(CustNo*, TypeID*)

/*

Part B

Problem 1

*/

```
SELECT b.title, b.subtitle, b.edition, b.place
FROM book b
INNER JOIN published_by pb
ON b.bookdescid = pb.bookdescid
```

```
WHERE LOWER(b.place) LIKE '%oxford%'
AND LOWER(role) LIKE 'publisher';
```

/*

Problem 2

*/

```
SELECT b.bookdescid, COUNT(bc.bookdescid) AS "number of copies"
FROM book b
INNER JOIN book_copy bc
ON b.bookdescid = bc.bookdescid
GROUP BY bc.bookdescid
ORDER BY "number of copies" DESC;
```

/*

Problem 3_A

*/

```
SELECT DISTINCT b.bookdescid, b.title, b.subtitle, b.edition, b.place
FROM book b
JOIN book_copy bc
ON b.bookdescid = bc.bookid
LEFT JOIN borrow_copy brc
```

```
ON bc.bookid = brc.bookid
WHERE brc.transactionID IS NULL;
```

```
/*
```

Problem 3_B

```
*/
```

```
SELECT b.bookdescid, b.title, b.subtitle, b.edition, b.place
FROM book b
WHERE b.bookdescid IN (
    SELECT bc.bookid
    FROM book_copy bc
    WHERE bc.bookid NOT IN(
        SELECT brc.bookid
        FROM borrow_copy brc)
);
```

```
/*
```

Problem 4

```
*/
```

```
SELECT a.firstname, a.middlename, a.lastname, COUNT(w.bookdescid) AS 'booknumb'
FROM author a
INNER JOIN written_by w
ON a.authorid = w.authorid
WHERE w.role = 'Author'
GROUP BY a.authorid
HAVING booknumb >1 ;
```

```
/*
```

Problem 5

```
*/
```

```

SELECT b.title, DATE(br.returndate) AS 'returnDate',
       DATE(br.duedate) AS 'dueDate',
       julianday(br.returndate) - julianday(br.duedate) AS 'daysDelay'
FROM borrow br
JOIN borrow_copy brc
ON br.transactionid = brc.transactionid
JOIN book_copy bc
ON bc.bookid = brc.bookid
LEFT JOIN book b
ON bc.bookid = b.bookdescid
WHERE julianday(br.returndate) > julianday(br.duedate);

```

/*

Problem 6

*/

```

SELECT p.publisherfullname
FROM publisher p
LEFT JOIN published_by pb
ON p.publisherID = pb.publisherID
AND pb.role = 'Editor'
WHERE pb.publisherID IS NULL
ORDER BY p.publisherfullname;

```

/*

Problem 7

*/

```

SELECT a.firstname|| ' ' || a.lastname AS "author",
       b.title AS "bookTitle",
       s.subjecttype AS "bookSubject",

```

```

        b.year AS "publishedYear"
FROM author a
INNER JOIN written_by wb
ON a.authorID = wb.authorID
INNER JOIN book b
ON wb.bookdescID = b.bookdescID
INNER JOIN subject s
ON b.subjectID = s.subjectID
WHERE s.subjecttype = 'Artificial Intelligence';

```

/*

Problem 8 ISYS1055

*/

```

SELECT b.title AS "bookTitle",
       p.firstname || ' ' || p.lastname AS 'borrowerName',
       DATE(br.borrowdate) AS Date_of_Borrow
FROM book b
JOIN borrow_copy brc
ON b.bookdescid = brc.bookid
JOIN borrow br
ON brc.transactionid = br.transactionid
JOIN person p
ON br.personid = p.personid
WHERE p.personid IN (SELECT p.personid
                     FROM book b
                     JOIN borrow_copy brc
                     ON b.bookdescid = brc.bookid
                     JOIN borrow br
                     ON brc.transactionid = br.transactionid
                     JOIN person p

```



```
        ON br.personid = p.personid
        WHERE LOWER(b.title) LIKE 'computer science')
AND LOWER(b.title) != 'computer science';
```

```
/*
```

Problem 8 ISYS3412

```
*/
```

```
SELECT b.bookdescid, b.title, b.year,
       COUNT(brc.transactionid) AS "timesBorrowed"
FROM book b
LEFT JOIN book_copy bc
  ON bc.bookid = b.bookdescid
LEFT JOIN borrow_copy brc
  ON brc.bookid = bc.bookid
GROUP BY b.bookdescid
HAVING COUNT(brc.transactionid) > 0
ORDER BY "timesBorrowed" DESC;
```